噪声增强对手写数字识别鲁棒性的影响

洪恩浩 | 学号 2026439132 | 深度学习课程设计 2026 秋季

实验日期 2026 年 11 月 10 日

代码仓库 GitHub https://github.com/hehcodeking/mnist-noise-robustness

摘要

手写数字识别模型在干净图像上表现良好，但实际输入中的噪声可能导致性能下降。本文在 MNIST 上研究一个具体问题：在模型结构与训练步数不变时，训练阶段混合干净图像与高斯噪声图像，能否提高识别鲁棒性？采用线性分类器和小型卷积神经网络，比较普通训练及两档噪声增强，在 3 个随机种子下进行重复实验。噪声标准差为 0.30 时，增强模型的平均准确率由 94.59% 提升至 97.07%；标准差为 0.50 时由 74.40% 提升至 92.86%。干净图像准确率为 97.72%，与普通 CNN 的 97.63% 接近。结果支持在本实验范围内采用噪声增强，但不能据此推断其在真实扫描件或所有噪声类型上均有效。

关键词 手写数字识别；卷积神经网络；数据增强；噪声鲁棒性

1 研究问题与应用价值

低质量扫描、拍摄和传输会改变图像像素。只报告干净测试集准确率，难以判断模型遇到质量下降时是否可靠。本文以单个、已对齐的数字图像为研究对象，将加性高斯噪声作为可控的像素扰动，检验一种计算成本较低的训练改进。普通扰动鲁棒性与对抗攻击防御属于不同问题，本实验采用前者的研究视角[3]。

研究假设是：在固定数据和训练预算下，噪声增强能提高中强噪声下的平均准确率，且不明显损害干净图像表现。进一步比较增强强度，并用未参与训练的椒盐噪声检查有限的跨扰动迁移。本文贡献是完成一个可核查的小型实验，而非提出新的网络结构。

2 数据来源与使用范围

使用 LeCun、Cortes 与 Burges 发布的 MNIST，经 CVDF 维护的镜像获取原始 IDX 文件 [1]。每张图像为 28×28 灰度图，类别为 0 至 9；原始训练集 60 000 张、官方测试集 10 000 张。所有像素除以 255 归一化到 [0,1]，不使用预训练权重。

表 1 数据划分与用途

数据部分	数量	用途
训练子集	12 000，每类 1 200	仅用于参数更新
验证子集	3 000，每类 300	选择每组最佳训练轮次
未使用训练数据	45 000	不参与训练或验证
官方测试集	10 000，保留原分布	固定模型后报告结果

训练与验证由固定种子 2026 分层抽样，索引无交集；检查发现验证及测试图像均无与训练子集像素完全一致的样本。实验结论限于上述数据规模。

3 方法设计与实现

3.1 网络结构及选择理由

线性基线将 784 个像素直接映射为 10 类，用于观察不具备局部特征提取能力的模型表现。CNN 以卷积核学习局部笔画特征，并通过池化压缩空间尺寸；结构较小，适合在 CPU 上重复训练。实现使用 PyTorch 的卷积、全连接、自动求导和优化器接口 [2]。

表 2 CNN 的层级结构

模块	输出尺寸	主要设置
输入	1×28×28	灰度像素归一化
卷积与池化	16×14×14	3×3 卷积，padding=1，ReLU，2×2 池化
卷积与池化	32×7×7	3×3 卷积，padding=1，ReLU，2×2 池化
展平与全连接	64	1 568 维输入，ReLU
分类输出	10	输出 logits，交叉熵内部处理

线性基线含 7 850 个可训练参数，CNN 含 105 866 个参数。三种 CNN 条件的结构、参数量和更新步数完全相同；不加入额外的去噪网络。

3.2 在线噪声增强

训练时对每张图像独立抽取伯努利变量 m，以 0.5 的概率添加高斯噪声，之后将像素截断到有效范围。增强在每个批次重新采样，保留原类别标签，不额外扩充批次或增加训练步数。像素变换如下：

$$\tilde{x} = clip(x + m\sigma\varepsilon, 0, 1)$$

其中 x 为归一化图像，ε 为逐像素独立的标准正态噪声，σ 控制增强强度，m 为取 0 或 1 的开关；clip 将数值截断到 [0,1]。m=0 时保留干净图像。所有模型均最小化类别交叉熵，即鼓励模型提高真实类别的预测概率。

3.3 对照和消融条件

表 3 固定训练预算下的四组设置

实验条件	训练扰动	比较目的
线性基线	无	基础分类能力参考
CNN 普通训练	无	检验卷积网络及无增强表现
CNN 增强 0.15	50% 图像加 σ=0.15 噪声	较弱增强的消融条件
CNN 增强 0.30	50% 图像加 σ=0.30 噪声	预设的主要改进方法

对同一随机种子的三个 CNN，固定相同初始权重和样本顺序，增强使用独立随机数生成器。这样能减少与增强本身无关的随机差异。代码分别实现数据校验与划分、模型构造、训练与选模、测试评价及图表生成，未复制第三方项目实现。

4 实验流程与评价方法

实验流程为：校验数据文件 → 固定训练与验证划分 → 按条件和种子训练 → 使用验证集选择权重 → 统一测试 → 保存逐样本预测并独立复算指标。训练脚本仅读取原始训练集的图像和标签；测试标签由评价脚本读取，不用于训练、选模或调整超参数。

表 4 训练配置与实际运行环境

项目	固定设置
软件环境	Python 3.12.14，PyTorch 2.5.1，NumPy 2.3.5
计算环境	macOS 26.6，ARM64 CPU，4 个计算线程
训练预算	每组 12 轮，batch size=128，每轮 94 次更新
优化方法	Adam，学习率 0.001，weight decay=0.0001
训练种子	11、22、33；CPU 确定性算法开启
权重选择	验证集干净准确率与 σ=0.30 准确率的均值最高
噪声随机源	验证 1701；测试 1702；两者相互独立

每组完成全部 12 轮，再取验证分数最高的一轮；相同分数保留较早轮次。使用同一选模规则，便于比较固定计算预算内各方法的行为。线性基线未单独调优，因此其结果仅为本设置下的参考，不代表线性方法的最佳性能。

4.1 测试任务与指标定义

高斯噪声测试取 σ=0、0.10、0.20、0.30、0.40、0.50；所有模型使用同一测试噪声，强度之间复用同一底噪以便配对比较。额外采用椒盐噪声：每个像素以总概率 0.10 被改为 0 或 1，两种取值各占一半。训练和验证均不使用椒盐噪声。每个条件评价全部 10 000 张官方测试图像，共得到 84 组记录。

准确率为正确预测数除以总样本数；宏平均 F1 先分别计算各类别的 F1，再对 10 类等权平均，以检查类别差异。主结果报告 3 次训练的均值±样本标准差；图中阴影是训练种子标准差，不是置信区间。另对同一测试图像的预测差异做 2 000 次配对 bootstrap，估计固定模型条件下的准确率差异区间。

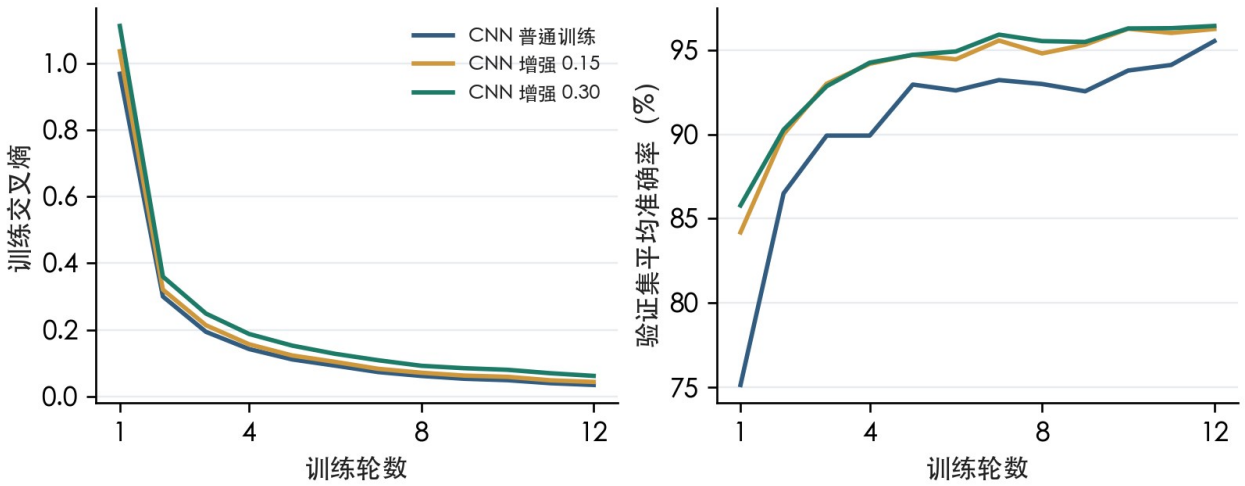


图 1 种子 11 的训练交叉熵与验证平均准确率。增强改变了训练数据难度，损失数值不宜直接视作模型优劣排名。

模型和数据的随机源均记录在配置中。相同环境下可重复训练；跨设备或库版本的数值仍可能存在差异 [4]。本报告未将训练耗时作为性能基准。

5 定量结果与有效性验证

表 5 汇总四组模型在主要测试条件下的表现。准确率与宏平均 F1 均以百分数表示；每个“±”后的数值为 3 个训练种子的样本标准差。

表 5 官方测试集上的主要结果

方法	干净 准确率	高斯 0.30 准确率	高斯 0.50 准确率	椒盐 0.10 准确率	高斯 0.30 宏 F1
线性基线	88.87±1.33	80.51±1.54	56.99±5.36	87.17±0.73	80.79±1.42
CNN 普通训练	97.63±0.04	94.59±1.29	74.40±5.12	93.88±1.27	94.53±1.33
CNN 增强 0.15	97.55±0.16	96.35±0.25	87.72±0.79	95.85±0.43	96.33±0.25
CNN 增强 0.30	97.72±0.24	97.07±0.29	92.86±0.51	96.76±0.30	97.06±0.29

5.1 噪声增强改善中强扰动下的表现

普通 CNN 在干净图像上达到 97.63%，但噪声增强到 σ=0.50 时降至 74.40%。主要方法在 σ=0.30 时达到 97.07%，相对普通 CNN 提升 2.48 个百分点，对应错误率由 5.41% 降至 2.93%，相对下降 45.90%；在 σ=0.50 时提升 18.46 个百分点。图 2 显示，增强对较强高斯噪声的收益更明显。

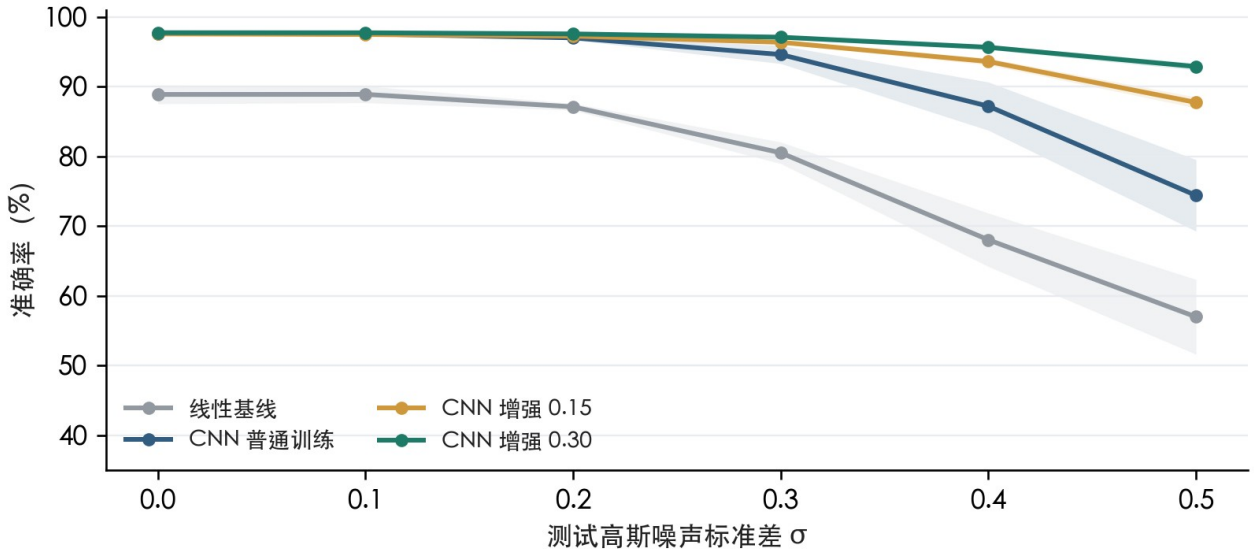


图 2 高斯噪声强度与识别准确率。点为 3 个训练种子的均值，阴影为±1 个样本标准差；纵轴未从 0 开始。

5.2 干净图像与未见扰动的结果

增强 0.30 的干净准确率为 97.72%，相对普通 CNN 仅增加 0.09 个百分点；该差异小于增强模型的种子标准差，且种子 22 的干净准确率下降 0.21 个百分点。因此只能认为本实验未观察到明显的平均性能损失，不能声称干净识别能力稳定提高。

在未参与训练的椒盐噪声下，准确率由 93.88% 提高至 96.76%，提升 2.89 个百分点。这表明高斯增强在该项测试中具有一定迁移收益，但尚不足以支持其对模糊、旋转或真实扫描噪声的普遍有效性。

6 消融实验与错误案例

6.1 增强强度及重复实验

将训练噪声从 0.15 提高到 0.30，高斯 0.30 测试准确率从 96.35% 升至 97.07%，高斯 0.50 测试从 87.72% 升至 92.86%。在本次比较的两档强度中，较强增强的抗噪收益更大。由于未测试更强训练噪声及其他混合概率，不能据此将 0.30 宣称为全局最优参数。

表 6 高斯 0.30 条件下普通 CNN 与增强 0.30 的配对比较

训练种子	普通 CNN	增强 0.30	差异 百分点	差异的 95% 区间 百分点
11	94.84%	97.04%	+2.20	[1.81, 2.60]
22	93.19%	96.80%	+3.61	[3.18, 4.05]
33	95.74%	97.38%	+1.64	[1.33, 1.96]

三个种子的噪声准确率均提高。区间通过对测试图像有放回配对抽样获得，条件是已训练权重和固定噪声；它不覆盖重新抽取训练集、重训模型或改变噪声随机源带来的不确定性。只有 3 次初始化和一个数据划分，也限制了外推强度。

6.2 成功修正与仍然失败的样本

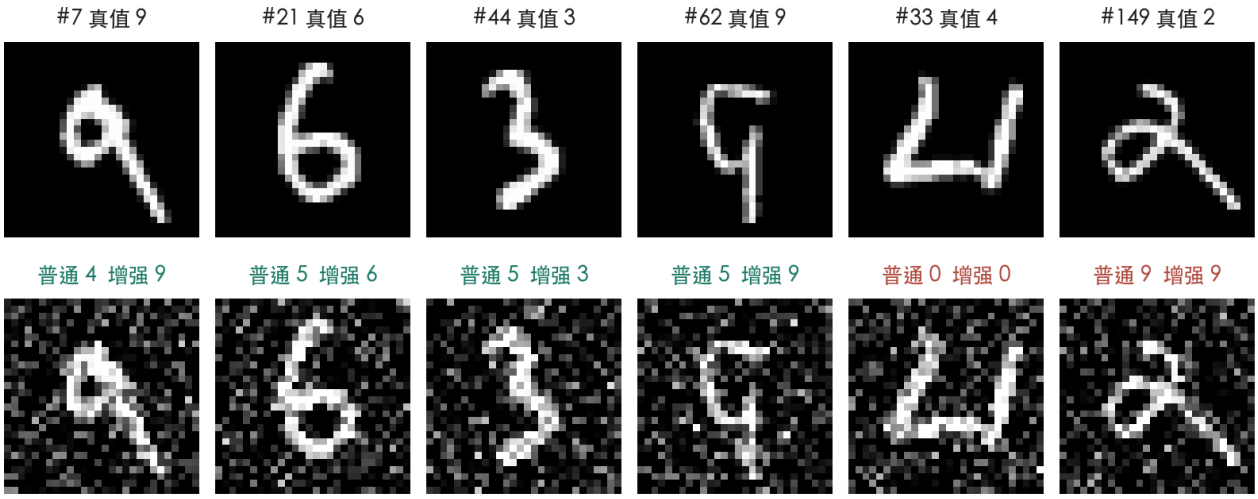


图 3 种子 11、σ=0.30 的真实测试案例。上排为原图，下排为加噪图；编号为从 0 开始的官方测试索引。前四列是按索引顺序最先出现的修正样本，后两列是增强模型最先出现的错误样本。

噪声改变笔画和背景后，普通 CNN 把 #7 的 9 预测为 4，把 #21 的 6 预测为 5；增强模型将其修正。对同一条件下全部 10 000 张图像统计，增强模型修正了普通模型的 312 个错误，同时使原本正确的 92 张图像变错，净增加 220 张正确识别，说明收益并非每个样本都一致。

增强模型仍把 #33 的 4 识别为 0，把 #149 的 2 识别为 9。原始字形本身具有不规则笔画，噪声增强无法替代更丰富的字形学习。按类别分析，数字 9 的召回率由 84.64% 升至 95.64%，数字 6 由 90.81% 升至 97.60%；数字 5 却从 99.44% 降至 96.97%。因此，整体准确率提升仍可能伴随局部类别退化。

7 讨论与结论

本实验支持研究假设的抗噪部分：在相同 CNN 结构、数据量和更新次数下，在线高斯增强提高了中强高斯噪声下的准确率，也降低了较强噪声条件下不同初始化的结果波动。可能的解释是，训练中出现了受扰动的笔画与背景，模型需要学习对这类像素变化更稳定的判别特征。该解释与结果一致，但本文未通过特征可视化直接验证其内部机制。

主要限制有三点。第一，MNIST 是已裁剪、居中的低分辨率数字图像；本文没有验证文本检测、分割、连续数字序列或真实表单识别。第二，仅使用 12 000 张训练图像和一个固定划分；训练种子标准差不等于真实部署的不确定性。第三，评价仅含高斯和一种椒盐噪声设置，未覆盖模糊、压缩、光照、旋转和对抗扰动。当前结果不能直接作为部署性能保证。

后续可先增加多个数据划分和独立噪声重复，检验收益是否稳定；再在固定预算下测试更多噪声强度与混合比例，并加入旋转、模糊和真实采集图像。对类别 5 等出现退化的情况，应单独分析错误分布；如面向实际应用，还需评价置信度和拒识策略。

结论是：对本实验中的小型 CNN，在线混合高斯噪声是一种实现简单、无需增加参数或训练步数的有效改进。其价值主要体现在受扰动条件下，而干净图像上的小幅变化应谨慎解释。

8 复现材料与实现来源

随附项目包包含原始数据及校验清单、固定配置、全部 12 份模型权重、144 条逐轮训练记录、84 组测试指标、逐样本预测、bootstrap 结果和图表。按 README 执行训练、测试、核验和绘图命令即可重新生成结果；verify.py 会独立检查数据划分、权重选择、预测一致性和指标计算。CPU 训练总计 13 536 次参数更新。

网络、增强和实验组织代码由本项目编写，使用 PyTorch 公共接口，不使用预训练模型；框架用法参考官方教程 [2]，随机性设置参考官方说明 [4]。本研究借鉴常见扰动鲁棒性的评估思路 [3]，未使用 ImageNet-C 数据，也未复现其完整基准。代码、模型权重与报告公开于首页所列 GitHub 仓库，随附项目包可离线核查。

参考文献

[1] LeCun Y, Cortes C, Burges C. MNIST handwritten digit database. CVDF 数据镜像与 IDX 说明. https://github.com/cvdfoundation/mnist

[2] PyTorch. Quickstart tutorial. https://docs.pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html

[3] Hendrycks D, Dietterich T. Benchmarking Neural Network Robustness to Common Corruptions and Perturbations. ICLR, 2019. https://arxiv.org/abs/1903.12261

[4] PyTorch. Reproducibility. https://docs.pytorch.org/docs/stable/notes/randomness.html

AI 工具使用说明 使用 OpenAI Codex 辅助代码编写、实验运行、结果核验与报告排版。所有数值和案例均来自随附程序的实际运行记录；未将 AI 生成文本或他人算法表述为原创算法。